Dynamic Programming
(Longest Common Subsequence)

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

Brief Overview

- **Divide and Conquer**
- **Dynamic Programming**
  - ✓ Optimal structure
  - ✓ Optimal overlapping subproblems
- **Examples**
  - ✓ Fibonacci numbers
  - ✓ Matrix-chain multiplication
  - ✓ Longest common subsequence (Today's topic)

Longest Common Subsequence

LONGEST COMMON SUBSEQUENCE

- The sequence $Z = (B, C, A)$
  - ✓ is a *subsequence* of $X = (B, D, C, A, B, A)$.
  - ✓ is also a subsequence of $Y = (A, B, C, B, D, A, B)$.
  - ✓ is a ***common*** *subsequence* of X and Y.
  - ✓ is **NOT** a *longest common subsequence*
    because (B, D, A, B) is a longer common subsequence.

- Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$,
  - ✓ another sequence $Z = \langle z_1, z_2, \dots, z_k \rangle$ is a *subsequence* of X
    if there exists a strictly increasing sequence $\langle i_1, i_2, \dots, i_k \rangle$ of indices of X such that for all $j = 1,2, \dots, k$,
    we have $x_{i_j} = z_j$.

- Given two sequences X and Y,
  a sequence Z is a *common subsequence* of X and Y
  if Z is a subsequence of both X and Y.

LCS (LONGEST COMMON SUBSEQUENCE)

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

Y=<A,B,C,B,D,A,B>

X=<B,D,C,A,B,A>

BCBA

Y=<A,B,C,B,D,A,B>

X=<B,D,C,A,B,A>

BDAB

Y=<A,B,C,B,D,A,B>

X=<B,D,C,A,B,A>

BCAB

There are 3 LCSs.

LCS (LONGEST COMMON SUBSEQUENCE)

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

Y=<A,B,C,B,D,A,B>

X=<B,D,C,A,B,A>

Y=<A,B,C,B,D,A,B>

X=<B,D,C,A,B,A>

Y=<A,B,C,B,D,A,B>

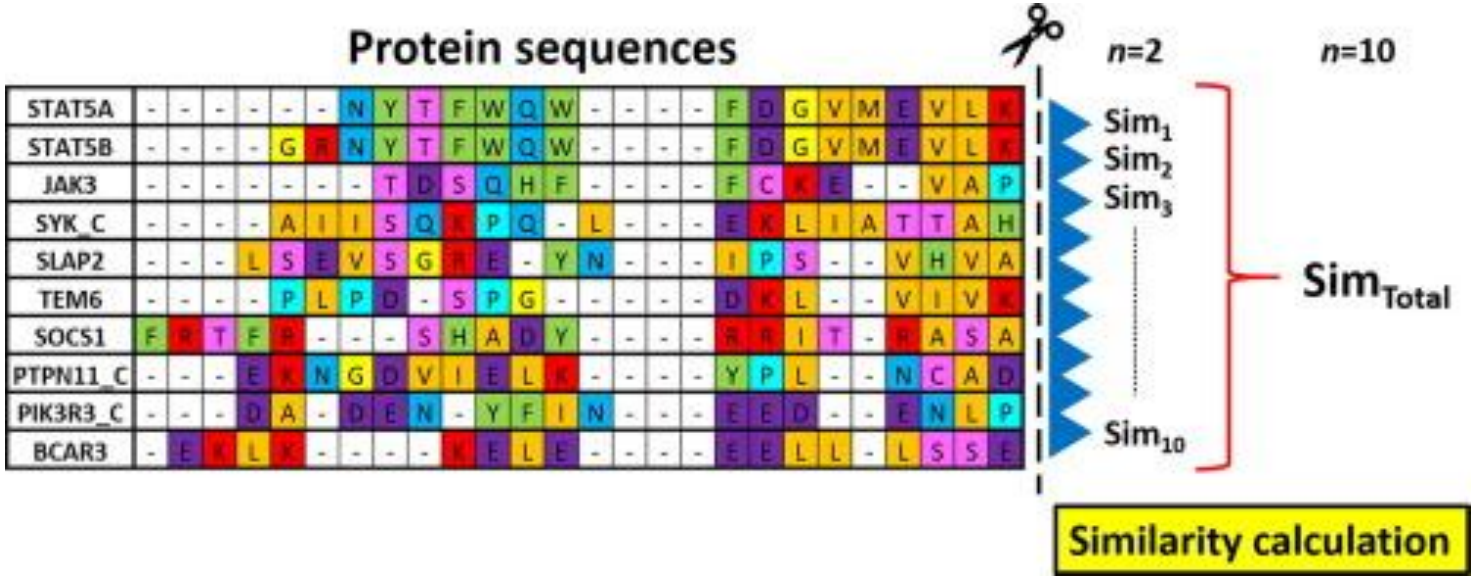
X=<B,D,C,A,B,A>

	A	B	C	B	D	A	B
B		X		X			X
D					X		
C			X				
A	X					X	
B				X			X
A	X					X	

LCS (LONGEST COMMON SUBSEQUENCE)

❖ Applications

- ✓ Plagiarism and Text Similarity Detection
  - LCS helps measure how much two sentences or documents share the same sequence of words or characters.
- ✓ Bioinformatics
  - LCS can be used to compare DNA or protein sequences and find similar patterns in biological data.
- ✓ User Behavior Analysis
  - LCS can be used to compare sequences of user actions, such as clicks or app usage patterns.
    - » User A: Home → Search → Product → Cart → Pay
    - » User B: B: Home → Search → Product → Wishlist → Cart → Pay



BRUTE-FORCE ALGORITHM

- Examine every subsequence of $X[1 \ldots m]$ to see if it is contained in $Y[1 \ldots n]$.
- Analysis
  - ✓ Number of subsequences of X is 2^m.
  - ✓ Time to check each against Y is $O(n)$. (Why?)
  - ✓ Thus, the worst-case running time is $O(n2^m)$.
  - ✓ So, the brute-force algorithm is not good.

LCS PROBLEM

- LCS problem:
  - ✓ Given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ find a longest subsequence common to both X and Y.
    - Value to be evaluated: length of LCS ($c[i,j]$)
    - Indicator: how to find the LCS ($\mathrm{b}[i,j]$: ↑,←,↖)

- A new useful definition
  - ✓ Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$, let's define the "$i$-th prefix of X" as $X_i = \langle x_1, x_2, \dots, x_i \rangle$ for $i = 0,1, \dots m$.
  - ✓ Example) $X = < A, B, C, B, D, A, B >$
    - $X_4 = < A, B, C, B >$
    - X_0 : empty sequence

STEP 1 : THEOREM 15.1 (IN THE TEXTBOOK)

❖ Step 1 : Theorem

✓ Given $X = \langle x_1, x_2, \dots, x_i \rangle$ and $Y = \langle y_1, y_2, \dots, y_j \rangle$,
and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be an LCS of X and Y.

1. If $x_i = y_j$, then $z_k = x_i = y_j$ and Z_{k-1} is an LCS of X_{i-1} and Y_{j-1}.
2. If $x_i \neq y_j$, then $z_k \neq x_i$ implies that Z is an LCS of X_{i-1} and Y.
3. If $x_i \neq y_j$, then $z_k \neq y_j$ implies that Z is an LCS of X and Y_{j-1}.

$$X = \langle x_1, x_2, \dots, x_i \rangle \qquad X_{i-1} = \langle x_1, x_2, \dots, x_{i-1} \rangle$$

$$Y = \langle y_1, y_2, \dots, y_j \rangle \qquad Y_{j-1} = \langle y_1, y_2, \dots, y_{j-1} \rangle$$

$$Z = \langle z_1, z_2, \dots, z_k \rangle \qquad Z_{k-1} = \langle z_1, z_2, \dots, z_{k-1} \rangle$$

STEP 2: A RECURSIVE SOLUTION

❖ Step 2 : Recursive solution

✓ c_{ij}= the length of an LCS of the sequences X_i and Y_j

$$c_{ij} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c_{i-1,j-1} + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c_{i,j-1}, c_{i-1,j}) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

1. If $x_i = y_j$, then $z_k = x_i = y_j$ and Z_{k-1} is an LCS of X_{i-1} and Y_{j-1}.
2. If $x_i \neq y_j$, then $z_k \neq x_i$ implies that Z is an LCS of X_{i-1} and Y.
3. If $x_i \neq y_j$, then $z_k \neq y_j$ implies that Z is an LCS of X and Y_{j-1}.

STEP 2: A RECURSIVE SOLUTION

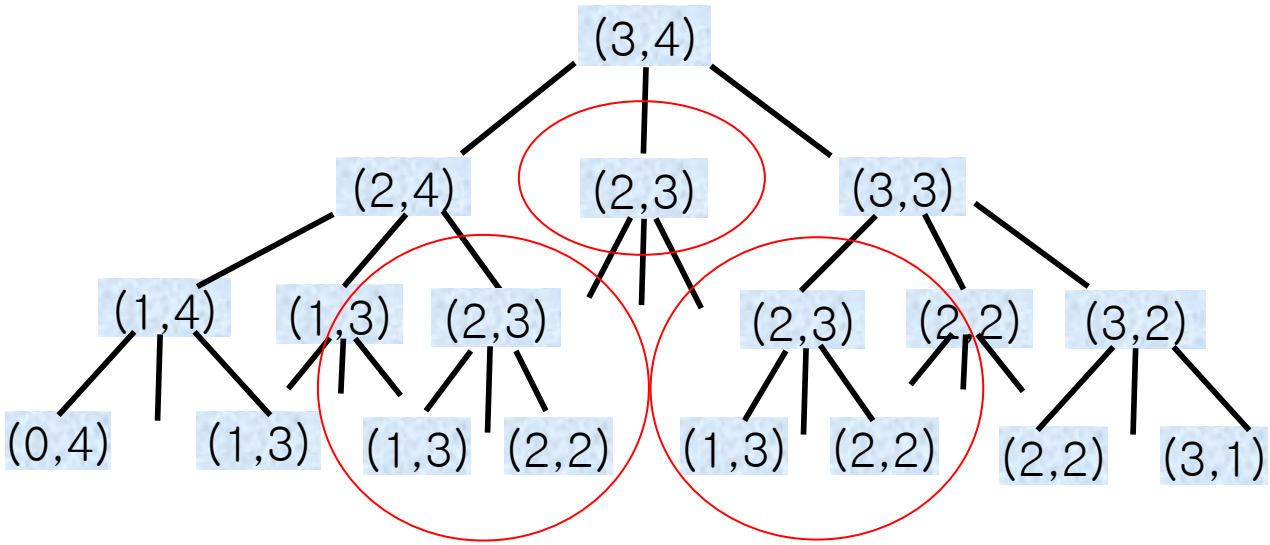
- **Step 2 : Recursive solution**
  - ✓ c_{ij}= the length of an LCS of the sequences X_i and Y_j

$$c_{ij} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c_{i-1,j-1} + 1 & \text{if } i,j > 0 \text{ and } x_i = y_j \\ \max(c_{i,j-1}, c_{i-1,j}) & \text{if } i,j > 0 \text{ and } x_i \neq y_j \end{cases}$$

```
LCS(X,Y,i,j) { // base case code is omitted
  if X[i]=Y[j]
    then c[i,j] ← LCS(X,Y,i-1,j-1)+1
  else  c[i,j] ← max{LCS(X,Y,i-1,j), LCS(X,Y,i,j-1)}
  return c[i,j]
}
```

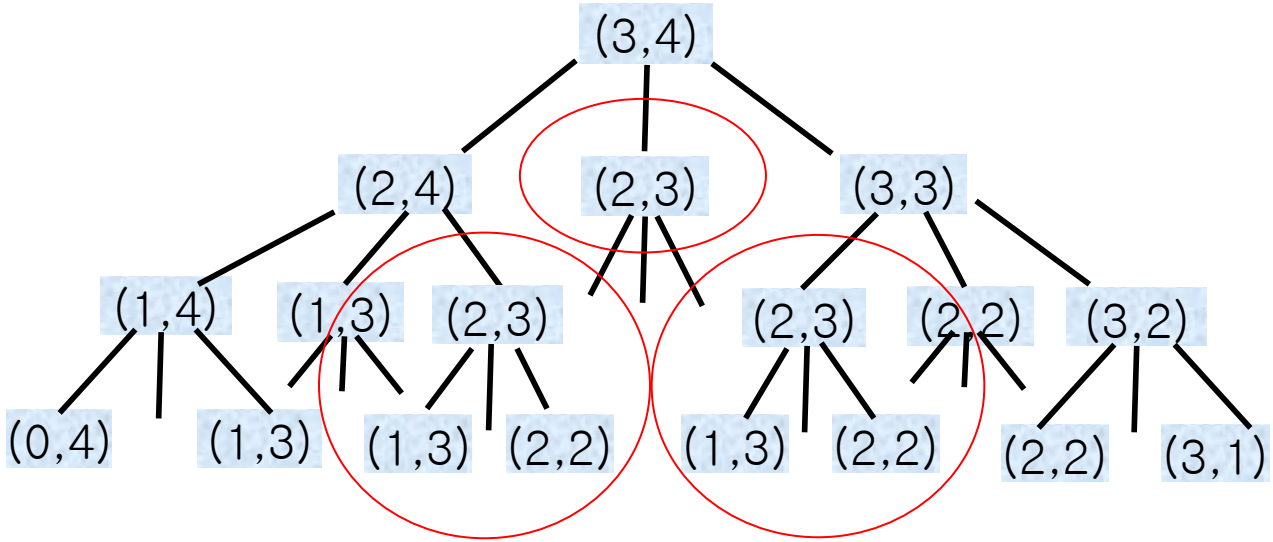
RECURSIVE ALGORITHM FOR LCS PROBLEM

```
LCS(X,Y,i,j) { // base case code is omitted
  if X[i]=Y[j]
    then c[i,j] ← LCS(X,Y,i-1,j-1)+1
  else  c[i,j] ← max{LCS(X,Y,i-1,j), LCS(X,Y,i,j-1)}
  return c[i,j]
}
```



Same subproblems!! → waste of time

RECURSIVE ALGORITHM FOR LCS PROBLEM



Overlapping subproblems:
A recursive solution contains distinct subproblems repeated many times.

→

After computing solutions to subproblems, store them in table to avoid re-doing work

Apply D.P.

STEP 3: COMPUTING LENGTH OF LCS

```
LCS-Length(X,Y) {
  m ← length[X]; n ← length[Y]
  for i ← 0 to m  c[i,0] ← 0
  for j ← 0 to n  c[0,j] ← 0
  for i ← 1 to m
    for j ← 1 to n
      if X[i] = Y[j]
        then c[i,j] ← c[i-1,j-1]+1; b[i,j] ← '↖'
      else
        if c[i-1,j] ≥ c[i,j-1]
          then c[i,j] ← c[i-1,j]; b[i,j] ← '↑'
        else c[i,j] ← c[i,j-1]; b[i,j] ← '←'
  return c and b
}
```

EXAMPLE

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

		0	1	2	3	4	5	6	7
			A	B	C	B	D	A	B
0		0	0	0	0	0	0	0	0
1	B	0							
2	D	0							
3	C	0							
4	A	0							
5	B	0							
6	A	0							

EXAMPLE

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

		0	1	2	3	4	5	6	7
			A	B	C	B	D	A	B
0		0	0	0	0	0	0	0	0
1	B	0	↑ 0	↖ 1	← 1	↖ 1	← 1	← 1	↖ 1
2	D	0							
3	C	0							
4	A	0							
5	B	0							
6	A	0							

EXAMPLE

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

		0	1	2	3	4	5	6	7
			A	B	C	B	D	A	B
0		0	0	0	0	0	0	0	0
1	B	0	0	1	1	1	1	1	1
2	D	0	0	1	1	1	2	2	2
3	C	0	0	1	2	2	2	2	2
4	A	0	1	1	2	2	2	3	3
5	B	0	1	2	2	3	3	3	4
6	A	0	1	2	2	3	3	4	4

Step 4 : Constructing an LCS

```
PRINT-LCS(b, X, i, j) {
  if i = 0 or j = 0
    then return
  if b[i, j] = '↖'
    then PRINT-LCS(b, X, i - 1, j - 1)
    print X[i]
  else if b[i, j] = '↑'
    then PRINT-LCS(b, X, i - 1, j)
  else
    then PRINT-LCS(b, X, i, j - 1)
}
```

EXAMPLE

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

		0	1	2	3	4	5	6	7
			A	B	C	B	D	A	B
0		0	0	0	0	0	0	0	0
1	B	0	0	1	1	1	1	1	1
2	D	0	0	1	1	1	2	2	2
3	C	0	0	1	2	2	2	2	2
4	A	0	1	1	2	2	2	3	3
5	B	0	1	2	2	3	3	3	4
6	A	0	1	2	2	3	3	4	4

BDAB

BCBA

EXAMPLE: COMPLEXITY

Let X=<B,D,C,A,B,A> and Y=<A,B,C,B,D,A,B>.

		0	1	2	3	4	5	6	7
			A	B	C	B	D	A	B
0		0	0	0	0	0	0	0	0
1	B	0	0	1	1	1	1	1	1
2	D	0	0	1	1	1	2	2	2
3	C	0	0	1	2	2	2	2	2
4	A	0	1	1	2	2	2	3	3
5	B	0	1	2	2	3	3	3	4
6	A	0	1	2	2	3	3	4	4

BDAB

BCBA

- **Space: Θ(mn)**
- **Time: Θ(mn)**

EXAMPLE: COMPLEXITY

BDAB BCBA

		0	1	2	3	4	5	6	7
			A	B	C	B	D	A	B
0		0	0	0	0	0	0	0	0
1	B	0	0	1	1	1	1	1	1
2	D	0	0	1	1	1	2	2	2
3	C	0	0	1	2	2	2	2	2
4	A	0	1	1	2	2	2	3	3
5	B	0	1	2	2	3	3	3	4
6	A	0	1	2	2	3	3	4	4

else **if** $c[i\text{-}1,j] \geq c[i,j\text{-}1]$

then $c[i,j] \leftarrow c[i\text{-}1,j]$; $b[i,j] \leftarrow \uparrow$

else $c[i,j] \leftarrow c[i,j\text{-}1]$; $b[i,j] \leftarrow \leftarrow$

	A	B	C	B	D	A	B
B		X		X			X
D					X		
C			X				
A	X					X	
B				X			X
A	X					X	

Quiz Announcement

QUIZ (3/27)

❖ Quiz (3/27) Announcement

- ✓ There will be a quiz on 3/27(Fri) during class (2:30 pm – 3:45 pm).
- ✓ On-site exam

- ✓ The quiz will consist of a combination of multiple-choice, true/false, and short/long-answer questions.
- ✓ Some questions may require you to write or draw your answers by hand and upload them to HDLMS, so please be prepared in advance.
  - You may prepare blank paper, a stylus-enabled tablet, or other appropriate devices.

- ✓ Open-lecture-slide, open-book exam
- ✓ NO online search, NO AI use

Thank you!